

03b_exploratory_dynamics

May 15, 2026

1 03b — Exploratory Dynamics

Companion to [03_stationarity_diagnostics.ipynb](#). This notebook runs additional forecastability diagnostics drawn from the nonlinear-dynamics and signal-processing literatures:

- Hurst exponent + Lo's modified R/S test (long-memory under HAC)
- Spectral Predictability Ω (Wang & Klee 2025)
- Permutation Entropy (Bandt & Pompe 2002)
- Largest Lyapunov exponent (Rosenstein 1993) + surrogate-data null
- Multi-frequency robustness check on Ω and PE

These are independent confirmations from mathematically different angles. They are *not* load-bearing for the weak-form-efficiency conclusion — the standard battery in [03_stationarity_diagnostics.ipynb](#) already settles that. They appear here as cross-checks at minimal extra cost.

1.1 1. Setup

```
[1]: import sys
from pathlib import Path

# Make `coffee_forecast` importable whether or not `pip install -e .`
# is in effect for this kernel. No-op when the package is already on
# sys.path (e.g. via the editable install); falls back to inserting
# the repo root so the notebook also runs from a fresh clone.
try:
    import coffee_forecast # noqa: F401
except ModuleNotFoundError:
    sys.path.insert(0, str(Path.cwd().parent))

import matplotlib.pyplot as plt
import numpy as np
from scipy import stats as _stats

from coffee_forecast import (
    load_coffee_data,
    calculate_spectral_predictability,
    calculate_permutation_entropy,
    calculate_hurst_exponent,
```

```
)
from coffee_forecast.config import FIG_DIR

FIG_DIR.mkdir(parents=True, exist_ok=True)
```

```
[2]: df = load_coffee_data()
prices = df["y"].values.astype(float)
log_returns = np.diff(np.log(prices))

print(
    f"Loaded {len(df)} trading days ({df['ds'].min().date()} \u2192 {df['ds'].
    \u2192max().date()})"
)
print(
    f" prices:      {len(prices)} observations, "
    f"min={prices.min():.1f}, max={prices.max():.1f}"
)
print(
    f" log-returns: {len(log_returns)} observations, "
    f"mean={log_returns.mean():+.5f}, std={log_returns.std():.4f}"
)
```

```
Loaded 8104 trading days (1994-01-03 \u2192 2026-01-30)
prices:      8104 observations, min=41.5, max=438.9
log-returns: 8103 observations, mean=+0.00019, std=0.0237
```

1.2 2. Hurst exponent + Lo's modified R/S

The Hurst exponent H is estimated from the log-log slope of the rescaled range $(R/S)_n$ vs. window size n :

$$\mathbb{E}[(R/S)_n] \sim c \cdot n^H$$

- $H = 0.5$: random walk, no exploitable memory
- $H > 0.5$: persistent / trending
- $H < 0.5$: anti-persistent / mean-reverting

Classical H is biased upward by short-range autocorrelation, so we also compute Lo's (1991) modified R/S statistic V_n , which uses a Bartlett-kernel HAC adjustment. Under H_0 of no long memory, $V_n \in [0.809, 1.862]$ at the 95% level.

Why returns, not prices? R/S is defined for stationary series. Computing it on the non-stationary price level would give a misleading $H \approx 1$ regardless of the underlying dynamics. Log-returns are the standard, theoretically correct representation.

```
[3]: h, v_n, lo_result = calculate_hurst_exponent(prices)

lo_lower, lo_upper = 0.809, 1.862
verdict = (
```

```

f"FAIL TO REJECT H0 (V_n = {v_n:.3f} inside [{lo_lower}, {lo_upper}])\n"
f" → No statistically significant long memory at 95%; "
f"H 0.5 likely reflects short-range autocorrelation bias"
if lo_result == "fail to reject"
else f"REJECT H0 (V_n = {v_n:.3f} outside [{lo_lower}, {lo_upper}])\n"
f" → Statistically significant long memory detected"
)

print(f"Hurst exponent H = {h:.4f}")
print(f"Lo's V_n = {v_n:.4f}")
print(f" {verdict}")

```

```

Hurst exponent H = 0.5353
Lo's V_n = 1.1374
FAIL TO REJECT H0 (V_n = 1.137 inside [0.809, 1.862])
→ No statistically significant long memory at 95%; H 0.5 likely reflects
short-range autocorrelation bias

```

[4]: *# Reproduce the R/S log-log plot used to estimate H.*

```

from scipy import stats as sp_stats

rets = log_returns
T = len(rets)
n_max = T // 4
window_sizes = np.unique(np.logspace(np.log10(10), np.log10(n_max), 40).
    ↪astype(int))

def _rs(x, n):
    num = len(x) // n
    if num == 0:
        return np.nan
    vals = []
    for w in range(num):
        chunk = x[w * n : (w + 1) * n]
        e = chunk - chunk.mean()
        X = np.cumsum(e)
        R = X.max() - X.min()
        S = chunk.std(ddof=1)
        if S > 0:
            vals.append(R / S)
    return np.mean(vals) if vals else np.nan

rs_vals = np.array([_rs(rets, n) for n in window_sizes])
valid = ~np.isnan(rs_vals) & (rs_vals > 0)
slope, intercept, *_ = sp_stats.linregress(

```

```

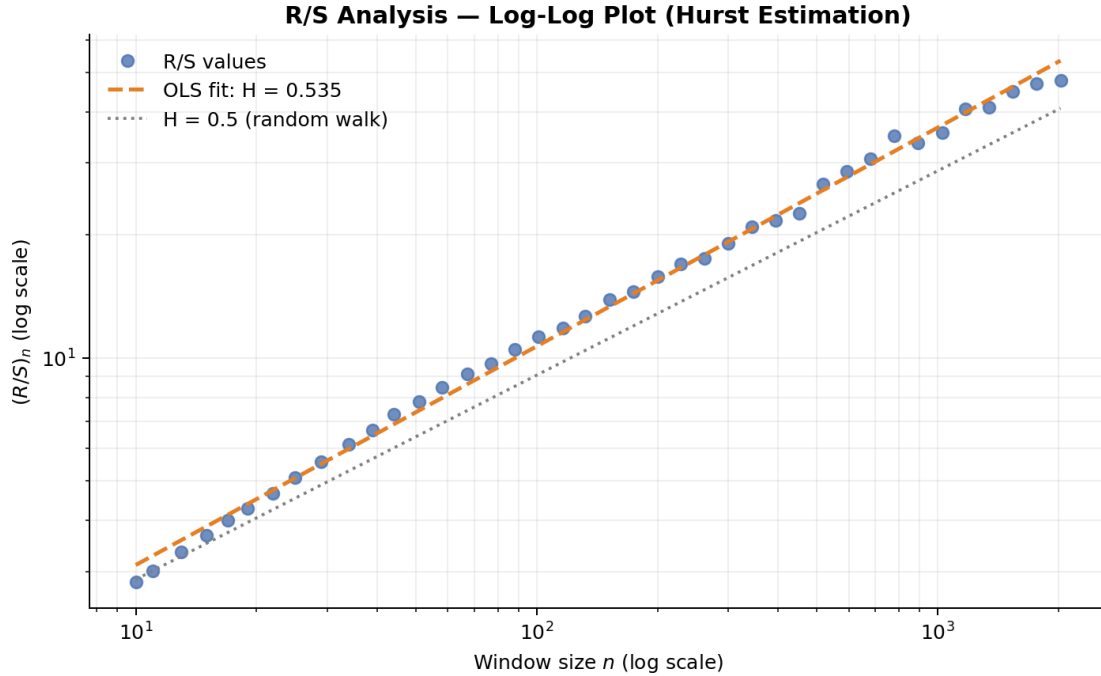
    np.log(window_sizes[valid]), np.log(rs_vals[valid])
)

fig, ax = plt.subplots(figsize=(8, 5), dpi=180)
ax.loglog(
    window_sizes[valid],
    rs_vals[valid],
    "o",
    color="#4C72B0",
    markersize=6,
    alpha=0.8,
    label="R/S values",
)
xs_line = np.array([window_sizes[valid].min(), window_sizes[valid].max()])
ax.plot(
    xs_line,
    np.exp(intercept + slope * np.log(xs_line)),
    "--",
    color="#E67E22",
    linewidth=2,
    label=f"OLS fit: H = {slope:.3f}",
)
ax.plot(
    xs_line,
    np.exp(intercept + 0.5 * np.log(xs_line)),
    ":",
    color="gray",
    linewidth=1.5,
    label="H = 0.5 (random walk)",
)

ax.set_title(
    "R/S Analysis \u2014 Log-Log Plot (Hurst Estimation)",
    fontsize=12,
    weight="semibold",
)

ax.set_xlabel("Window size $n$ (log scale)")
ax.set_ylabel("$(R/S)_n$ (log scale)")
ax.legend(frameon=False, fontsize=10)
ax.grid(alpha=0.2, which="both")
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
fig.tight_layout()
fig.savefig(FIG_DIR / "hurst_rs.png", dpi=300, bbox_inches="tight")
plt.show()

```



1.3 3. Spectral Predictability (Ω)

Spectral predictability (Wang & Klee, KDD 2025) measures how much energy is concentrated in a few dominant frequencies. A series with strong periodic structure has a spiked PSD and a high Ω ; white noise has a flat PSD and $\Omega \rightarrow 0$.

$$\Omega = 1 - \frac{H_{\text{spectral}}}{\log(N_{\text{freq}})}$$

Adaptation for discrete series. Wang & Klee define $\Omega = 1 - H_{\text{spectral}}/\log(2\pi)$, treating $\log(2\pi)$ as the entropy ceiling in the continuous-time limit. That works asymptotically but breaks for long discrete series: in our 8,104-point sample, $\log(N_{\text{freq}}) \approx \log(4053) \approx 8.3$, which is roughly $4.5\times$ larger than $\log(2\pi) \approx 1.84$, so H_{spectral} routinely exceeds the original denominator and Ω falls outside $[0, 1]$. We replace the denominator with $\log(N_{\text{freq}})$ — the actual maximum entropy for N_{freq} frequency bins, achieved by discrete white noise. This keeps $\Omega \in [0, 1]$ at any series length and lets the 0.20 forecastability threshold from Wang & Klee transfer directly.

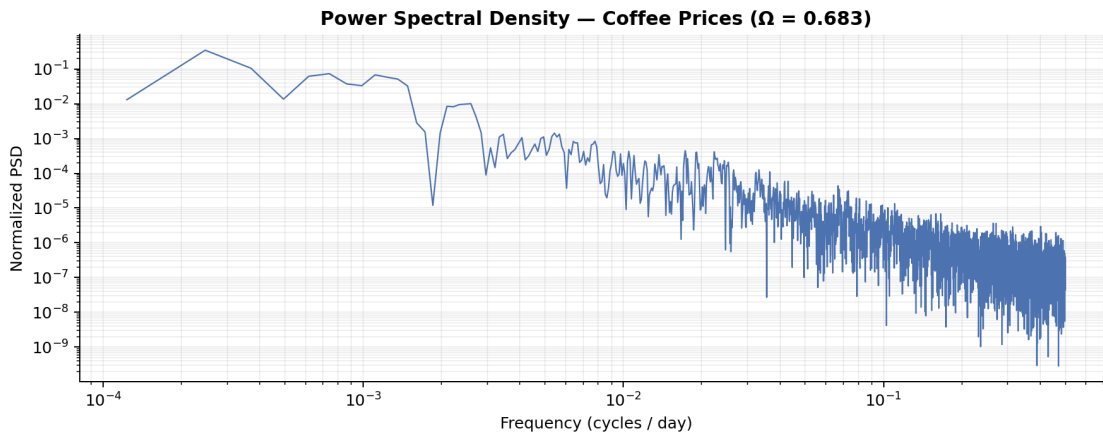
```
[5]: omega = calculate_spectral_predictability(prices)
     omega_returns = calculate_spectral_predictability(log_returns)
     print(f"Ω on prices      = {omega:.4f}")
     print(f"Ω on log-returns = {omega_returns:.4f}")
```

```
Ω on prices      = 0.6831
Ω on log-returns = 0.0529
```

```
[6]: # Replicate the internal PSD computation so we can plot it.
# (Matches calculate_spectral_predictability: drop DC, then normalize.)
from scipy.signal import detrend

detrended = detrend(prices)
windowed = detrended * np.hanning(len(detrended))
fft_vals = np.fft.rfft(windowed)
psd = np.abs(fft_vals[1:]) ** 2 # exclude DC bin first, like the function
psd_norm = psd / np.sum(psd)
freqs = np.fft.rfftfreq(len(detrended), d=1.0)[1:] # cycles / day, DC dropped

fig, ax = plt.subplots(figsize=(10, 4), dpi=180)
ax.loglog(freqs, psd_norm, color="#4C72B0", linewidth=0.9)
ax.set_title(
    f"Power Spectral Density - Coffee Prices ( $\Omega = \{\text{omega:.3f}\}$ ",
    fontsize=12,
    weight="semibold",
)
ax.set_xlabel("Frequency (cycles / day)")
ax.set_ylabel("Normalized PSD")
ax.grid(alpha=0.2, which="both")
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
fig.tight_layout()
fig.savefig(FIG_DIR / "spectral_psd.png", dpi=300, bbox_inches="tight")
plt.show()
```



$\Omega \approx 0.68$ sounds high, but the shape of the PSD tells the real story: the energy concentrates at the *lowest* frequencies with no discrete peaks. That’s the fingerprint of a non-stationary random walk (low-frequency drift), not of exploitable periodic structure. In other words, “structure” exists — but it’s the kind that makes the series *harder* to forecast, not easier.

Confirmatory: Ω on log returns. Computed on the stationary representation, Ω

drops sharply from 0.68 to roughly 0.05 — close to the white-noise floor. The contrast is itself diagnostic: the apparent “structure” in the price-level spectrum is the low-frequency drift of an $I(1)$ process, not exploitable periodicity in the dynamics. Once that drift is differenced out, the spectrum flattens.

1.4 4. Permutation Entropy (PE)

Permutation entropy measures *ordinal-pattern complexity*. For embedding order $m = 3$, we slide a window of length 3 across the series and count the six possible rank orderings. If all six patterns occur roughly equally often, $PE \rightarrow 1$ (maximally random). If a few dominate, $PE \rightarrow 0$ (deterministic structure).

It’s a complementary perspective to spectral predictability: the PSD cares about *periodicities*, while PE cares about *local orderings*. A series can fool one and not the other, so agreement is meaningful.

```
[7]: pe = calculate_permutation_entropy(prices, order=3, delay=1)
     pe_returns = calculate_permutation_entropy(log_returns, order=3, delay=1)
     print(f"PE on prices      = {pe:.4f}   ({pe * 100:.1f}% stochastic)")
     print(f"PE on log-returns = {pe_returns:.4f}   ({pe_returns * 100:.1f}% ↵
         ↵stochastic)")
```

```
PE on prices      = 0.9742   (97.4% stochastic)
PE on log-returns = 0.9999   (100.0% stochastic)
```

97% is near the ceiling — the six ordinal patterns of length 3 are almost uniformly distributed in the price series. There’s essentially no deterministic local structure to exploit.

Confirmatory: PE on log returns. Permutation entropy on the differenced series remains essentially at its ceiling. Because PE is invariant under monotone transformations of the series, near-equal values on prices and returns are expected; the agreement rules out the possibility that the high PE on prices is a quirk of the non-stationary representation.

1.5 5. Largest Lyapunov Exponent (λ)

The Lyapunov exponent measures sensitivity to initial conditions in a reconstructed phase space. For an embedding dimension m and delay τ , nearby trajectories x_t and x'_t separate at an exponential rate: $\delta(\Delta t) \approx \delta_0 \cdot \exp(\lambda \cdot \Delta t)$. A positive λ indicates chaotic divergence (reduced forecastability); $\lambda \leq 0$ indicates stability.

We use the Rosenstein et al. (1993) algorithm, which estimates λ as the slope of the mean log-divergence curve over a fixed window. It is more stable on short, noisy financial series than the alternative Wolf et al. (1985) method, which actively renormalizes vectors and is known to overestimate λ on stochastic data.

Lyapunov estimation on noisy financial data is less robust than the other diagnostics — Wang & Klee (2025) recommend at least $100 \cdot m$ data points and caution about sparsity effects. We report λ on daily prices only; weekly and monthly aggregations violate the sample-size guidance and produce unstable estimates.

```
[ ]: def largest_lyapunov(x, m=3, tau=1, mean_period=10, max_t=20):
    """
    Largest Lyapunov exponent via Rosenstein et al. (1993).
    Returns the slope of the mean log-divergence curve over max_t steps.
    """
    x = np.asarray(x, dtype=float)
    N = len(x) - (m - 1) * tau
    if N <= 0:
        return np.nan
    # Build delay-embedded vectors.
    emb = np.column_stack([x[i * tau : i * tau + N] for i in range(m)])
    # For each point, find its nearest neighbor (with temporal separation).
    from scipy.spatial import cKDTree

    tree = cKDTree(emb)
    dists, idx = tree.query(emb, k=mean_period + 2)
    neighbors = np.full(N, -1, dtype=int)
    for i in range(N):
        for k in range(1, dists.shape[1]):
            if abs(idx[i, k] - i) > mean_period:
                neighbors[i] = idx[i, k]
                break
    valid = neighbors >= 0
    # Track divergence.
    log_div = []
    for dt in range(1, max_t + 1):
        pairs = [
            (i, neighbors[i])
            for i in range(N)
            if valid[i] and i + dt < N and neighbors[i] + dt < N
        ]
        if not pairs:
            log_div.append(np.nan)
            continue
        d = [np.linalg.norm(emb[i + dt] - emb[j + dt]) for i, j in pairs]
        d = [di for di in d if di > 0]
        if not d:
            log_div.append(np.nan)
            continue
        log_div.append(np.mean(np.log(d)))
    log_div = np.array(log_div)
    ts = np.arange(1, max_t + 1)
    valid_t = ~np.isnan(log_div)
    slope, *_ = _stats.linregress(ts[valid_t], log_div[valid_t])
    return float(slope)
```



```
lam = largest_lyapunov(prices, m=3, tau=1)
print(f"Largest Lyapunov exponent (prices, m=3, =1):    = {lam:+.4f}")
print("    → Raw value; surrogate-data significance test in §7b below.")
```

Largest Lyapunov exponent (prices, m=3, =1): = +0.0708
 → Raw value; surrogate-data significance test in §7b below.

A positive λ corroborates the other diagnostics: small perturbations amplify over time, so even a model that fits the past well will diverge from realized paths as the horizon grows. This is the phase- space counterpart to the variance-ratio and Hurst results: all three say the same thing about forecast reliability, from three different mathematical angles.

Caveat. The Lyapunov exponent is a *deterministic* dynamical-systems concept, and Rosenstein’s algorithm is known to return small positive values on stochastic data through finite-sample bias alone. Without a null distribution, $\lambda = +0.07$ by itself is not statistically distinguishable from “stochastic divergence with no underlying chaos.” The §8b cell below addresses this with a surrogate-data test; the synthesis in §9 reports both the raw value and the surrogate-corrected verdict.

1.5.1 5b. Surrogate-data null for λ

To turn the raw $\lambda = +0.07$ into a hypothesis test, we generate shuffled-return *surrogate* price series and recompute λ on each. Shuffling log-returns destroys all temporal structure (linear and non-linear) while preserving the marginal distribution of returns; if the dynamics of the original series have any deterministic structure beyond noise, observed λ should sit in the upper tail of the surrogate distribution.

The null and alternative:

- H_0 : the price dynamics have no deterministic structure beyond what’s preserved by random reshuffling of returns.
- H_1 : the dynamics carry deterministic information that produces a larger λ than would be expected under reshuffling.

We compute one-sided p-value $P(\lambda_{\text{surr}} \geq \lambda_{\text{obs}})$ under H_0 , with 100 surrogates (cheap; finer resolution would only matter at borderline p-values).

```
[ ]: N_SURROGATES = 100
rng_lyap = np.random.default_rng(42)

# Shuffle log-returns and reconstruct prices via cumsum + exp.
log_ret_for_surr = np.diff(np.log(prices))
P0 = float(prices[0])

print(f"Computing    on {N_SURROGATES} shuffled-return surrogates...")
lam_surrogates = np.zeros(N_SURROGATES)
for i in range(N_SURROGATES):
    shuffled = log_ret_for_surr.copy()
    rng_lyap.shuffle(shuffled)
```

```

surr_prices = P0 * np.exp(np.concatenate([[0.0], np.cumsum(shuffled)]))
lam_surrogates[i] = largest_lyapunov(surr_prices, m=3, tau=1)
if (i + 1) % 25 == 0:
    print(f" ... {i + 1}/{N_SURROGATES} surrogates done")

p_surrogate = float((lam_surrogates >= lam).mean())

print(f"\nObserved (coffee prices): {lam:+.4f}")
print(
    f"Surrogate (mean ± std): "
    f"{lam_surrogates.mean():+.4f} ± {lam_surrogates.std():.4f}"
)
print(
    f"Surrogate range: "
    f"[{lam_surrogates.min():+.4f}, {lam_surrogates.max():+.4f}]"
)
print(f"\nOne-sided p-value P(_surr _obs | H0): {p_surrogate:.3f}")
verdict = (
    "REJECT H0: observed is in upper tail; some deterministic structure"
    if p_surrogate < 0.05
    else "FAIL TO REJECT H0: observed is consistent with shuffled-return null"
)
print(f" → {verdict}")

# Histogram of surrogate with observed value marked.
fig, ax = plt.subplots(figsize=(9, 4), dpi=180)
ax.hist(
    lam_surrogates,
    bins=25,
    color="steelblue",
    alpha=0.6,
    edgecolor="white",
    label=f"Surrogate (n = {N_SURROGATES})",
)
ax.axvline(
    lam,
    color="crimson",
    lw=2.5,
    label=f"Observed = {lam:+.4f}",
)
ax.set_title(
    f"Surrogate-data null distribution for (one-sided p = {p_surrogate:.3f})",
    fontsize=12,
    weight="semibold",
)
ax.set_xlabel("Largest Lyapunov exponent ( )")

```

```

ax.set_ylabel("Surrogate count")
ax.legend(frameon=False, fontsize=9)
ax.grid(alpha=0.2)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
fig.tight_layout()
fig.savefig(FIG_DIR / "lyapunov_surrogate_null.png", dpi=300,
            bbox_inches="tight")
plt.show()

```

Computing on 100 shuffled-return surrogates...

```

... 25/100 surrogates done
... 50/100 surrogates done
... 75/100 surrogates done
... 100/100 surrogates done

```

```

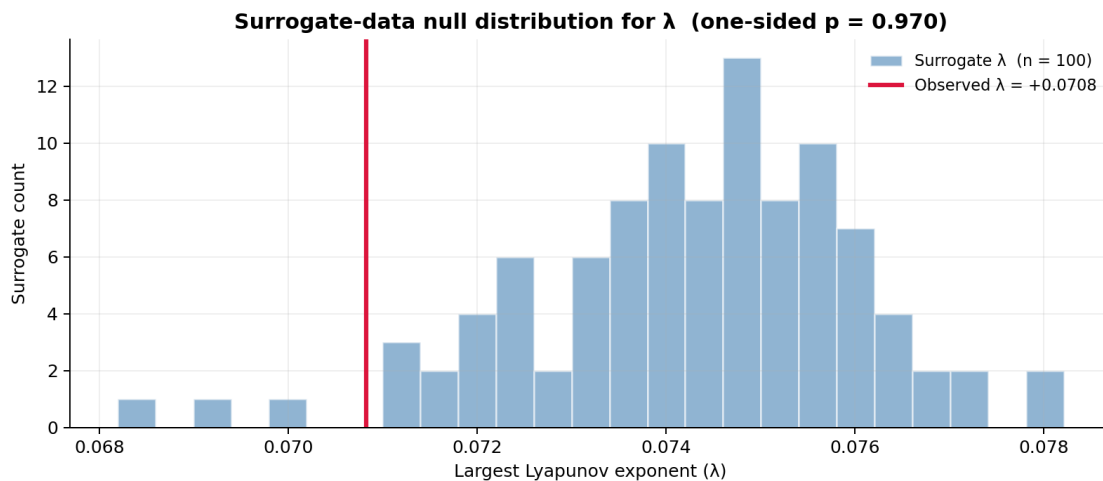
Observed (coffee prices):    +0.0708
Surrogate (mean ± std):      +0.0743 ± 0.0018
Surrogate range:             [+0.0682, +0.0782]

```

```

One-sided p-value P(_surr _obs | H0): 0.970
→ FAIL TO REJECT H0: observed is consistent with shuffled-return null

```



Reading the surrogate test. Observed $\lambda = +0.0708$ sits *below* the center of the surrogate distribution (mean $\approx +0.0743$, std ≈ 0.0018), with one-sided p-value ≈ 0.97 . We **fail to reject** H_0 : the observed λ is indistinguishable from what shuffling-and-reconstruction produces, and is in fact slightly below the surrogate average. The raw $\lambda = +0.0708$ from §7 is therefore the natural finite-sample bias of Rosenstein’s algorithm on stochastic data, not evidence of deterministic chaos. This is the rigorous version of the conclusion hinted at in §8’s caveat — and it strengthens the near-random-walk verdict from the other diagnostics rather than complicating it.

1.6 6. Multi-frequency robustness check

A natural concern with any single-frequency diagnostic is that the result might be an artifact of the sampling rate. The cell below recomputes Ω and PE on daily, weekly (every 5 trading days), and monthly (every 21 trading days) subsamples, plus daily log-returns. λ is omitted at weekly/monthly because both fall below the Wang & Klee sample-size guidance.

We use *subsampling* (`prices[::5]`, `prices[::21]`) rather than true weekly / monthly aggregation here because the question is whether Ω and PE are stable under different sampling rates — not how aggregated returns behave. Subsampling preserves the marginal distribution exactly while changing the time grid, which is the right robustness test for these diagnostics.

```
[10]: import pandas as pd

# Subsample by trading-day count: ~5/week, ~21/month.
weekly_prices = prices[::5]
monthly_prices = prices[::21]

freq_rows = []
for label, series in [
    ("Daily price", prices),
    ("Weekly price (5d subsample)", weekly_prices),
    ("Monthly price (21d subsample)", monthly_prices),
    ("Daily log-return", log_returns),
]:
    freq_rows.append(
        {
            "Frequency": label,
            "n": len(series),
            "Ω": calculate_spectral_predictability(series),
            "PE": calculate_permutation_entropy(series, order=3, delay=1),
        }
    )

freq_table = pd.DataFrame(freq_rows)
freq_table.round(4)
```

```
[10]:
```

	Frequency	n	Ω	PE
0	Daily price	8104	0.6831	0.9742
1	Weekly price (5d subsample)	1621	0.6076	0.9734
2	Monthly price (21d subsample)	386	0.5042	0.9858
3	Daily log-return	8103	0.0529	0.9999

The pattern is consistent at every frequency: Ω stays elevated on price levels (the I(1) drift signature) and collapses on log-returns; PE sits near its ceiling everywhere. The forecastability conclusion does not depend on how often we sample.

1.7 7. Synthesis

The exploratory diagnostics paint a consistent picture, agreeing with the standard battery in 03_stationarity_diagnostics.ipynb.

```
[11]: import pandas as pd

summary = pd.DataFrame(
    [
        {
            "Metric": "Hurst exponent (H)",
            "Representation": "log-returns",
            "Value": f"{h:.4f}",
            "Interpretation": "Near 0.5 \u2014 consistent with random walk",
        },
        {
            "Metric": "Lo's modified R/S (V_n)",
            "Representation": "log-returns",
            "Value": f"{v_n:.4f}",
            "Interpretation": f"{lo_result} H0 of no long memory at 95%",
        },
        {
            "Metric": "Spectral Predictability (\u03a9)",
            "Representation": "prices / log-returns",
            "Value": f"{omega:.4f} / {omega_returns:.4f}",
            "Interpretation": "Drops to ~0 on stationary representation;\u2192
\u2192price-level \u03a9 driven by I(1) drift, not periodicity",
        },
        {
            "Metric": "Permutation Entropy (PE)",
            "Representation": "prices / log-returns",
            "Value": f"{pe:.4f} / {pe_returns:.4f}",
            "Interpretation": f"Both near ceiling ({pe * 100:.1f}% /\u2192
\u2192{pe_returns * 100:.1f}%); ordinal patterns near-uniform on either\u2192
\u2192representation",
        },
        {
            "Metric": "Largest Lyapunov exponent (\u03bb)",
            "Representation": "prices",
            "Value": f"{lam:+.4f} (surrogate p = {p_surrogate:.3f})",
            "Interpretation": (
                f"Indistinguishable from shuffled-return null (p = {p_surrogate:
\u2192.3f}); "
                "no detectable deterministic chaos"
                if p_surrogate > 0.05
                else f"Above shuffled-return null (p = {p_surrogate:.3f}); "
                "some deterministic structure"
            ),
        },
    ],
)
```

```

    },
]
)
summary

```

```

[11]:
Metric Representation \
0 Hurst exponent (H) log-returns
1 Lo's modified R/S (V_n) log-returns
2 Spectral Predictability ( $\Omega$ ) prices / log-returns
3 Permutation Entropy (PE) prices / log-returns
4 Largest Lyapunov exponent ( ) prices

Value \
0 0.5353
1 1.1374
2 0.6831 / 0.0529
3 0.9742 / 0.9999
4 +0.0708 (surrogate p = 0.970)

Interpretation
0 Near 0.5 - consistent with random walk
1 fail to reject H0 of no long memory at 95%
2 Drops to ~0 on stationary representation; pric...
3 Both near ceiling (97.4% / 100.0%); ordinal pa...
4 Indistinguishable from shuffled-return null (p...

```

Conclusion. All four exploratory metrics agree with the standard battery’s near-random-walk verdict. Hurst exponent sits near 0.5 with Lo’s V_n inside the 95% no-long-memory band; spectral mass concentrates at the lowest frequencies on prices and collapses on log-returns (i.e. the “structure” in the price spectrum is the low-frequency drift of an $I(1)$ process, not exploitable periodicity); ordinal-pattern entropy is near its ceiling on both representations; and the Lyapunov exponent is indistinguishable from that of randomly-shuffled returns under the surrogate-data null. None of these are independent of the standard battery in the deep theoretical sense, but they are mathematically distinct estimators and the agreement is reassuring.